How the Freestyle Ray Tracer Works

A visual walk through primary, shadow, reflected and refracted rays

Abstract

This tutorial explains, step by step, how the freestyle ray tracer turns a scene description into an image. It deliberately avoids talking about code and variable names: every stage is described as geometry and arithmetic, illustrated with a figure and summarised with a short algorithm. The focus is on the life of a *ray*: how it is born at the camera, how it finds the surface it hits, how *shadow rays* decide whether that surface is lit, and how *reflected* and *refracted* rays carry the picture further. The last section assembles everything into one complete algorithm.

Contents

1 The big picture **2**

2 Primary rays: from pixel to ray **2**

3 Finding the nearest hit **3**
3.1 Cheap rejection with bounding boxes . . . 4
3.2 Exact intersection per shape . . . 4

4 When a ray hits nothing: background and glow **5**
4.1 Background . . . 5
4.2 Glow halos around emissive spheres . . . 6

5 Direct lighting: the Phong model **7**

6 Shadow rays **8**
6.1 The idea . . . 8
6.2 Avoiding shadow acne . . . 9
6.3 How dark is the shadow? (soft-shadow heuristic) . . . 9

7 Reflected rays **10**

8 Refracted rays **12**
8.1 Snell's law . . . 12
8.2 Total internal reflection . . . 12
8.3 Entering or leaving? Orienting the normal and finding n_1, n_2 . . . 13
8.4 How much is reflected? Fresnel and Schlick . . . 13
8.5 Tint . . . 13

9 Blending everything at a hit point **14**

10 Post-processing: gamma correction **15**

11 Putting it all together **15**

A Notes from reading the code **17**

1 The big picture

A ray tracer runs the physics of light *backwards*. Rather than following the billions of photons a lamp emits and hoping a few reach the eye, it starts at the eye, shoots one ray through every pixel and asks: *what colour of light arrives along this line?* To answer, it finds the first surface the ray hits and then shoots more rays from there:

- a **shadow ray** towards each light, to check whether the point is lit;
- a **reflected ray**, if the surface is shiny or transparent;
- a **refracted ray**, if the surface lets light through.

The reflected and refracted rays are handled exactly like the original ray, so the whole process is *recursive*. A depth budget stops it eventually. Figure 1 shows all four kinds of ray at once.

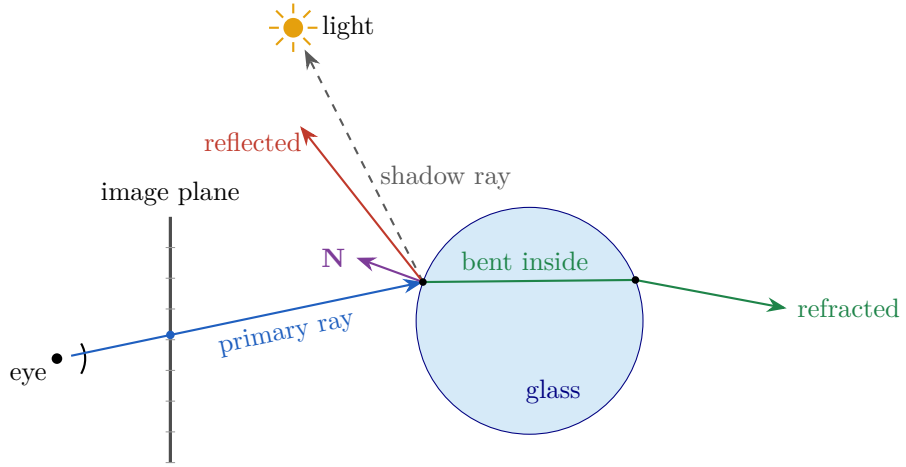


Figure 1: One pixel's story. The primary ray (blue) leaves the eye through a pixel and hits a glass sphere. From the hit point a shadow ray (dashed) checks the light, a reflected ray (red) bounces off and a refracted ray (green) bends into the sphere, bends again on the way out and continues into the scene.

A ray is a half-line with an origin $\mathbf{o}$ and a unit direction $\widehat{\mathbf{d}}$:

$$\mathbf{r}(t) = \mathbf{o} + t\,\widehat{\mathbf{d}}, \qquad t > 0.$$

Because the direction has unit length, the parameter t is exactly the distance travelled. "Nearest hit" therefore just means "smallest positive t".

Colours are 8-bit RGB triples. Every stage below produces a colour, and colours from different rays are blended with weights between 0 and 1.

2 Primary rays: from pixel to ray

The camera is a *pinhole*. All rays start at a single centre of projection $\mathbf{C}$ and pass through a flat image plane in front of it at distance f (the camera depth). The size of that plane follows from the horizontal and vertical fields of view ϕ_x, ϕ_y:

$$W = 2f \tan\frac{\phi_x}{2}, \qquad H = 2f \tan\frac{\phi_y}{2}.$$

W and H also set the image resolution in pixels, so a deeper camera with the same field of view gives a larger image.

Placing the plane in the world. The plane is first built in camera coordinates, where the camera looks along the z axis and the plane sits at $z = f$. Its four corners are $(\pm W/2,\ \pm H/2,\ f)$. They are taken to world space by undoing the camera rotation (for a rotation matrix the inverse is the transpose) and adding the camera position:

$$\mathbf{p}_{\text{world}} = \mathbf{C} + R^{\top}\mathbf{p}_{\text{cam}}.$$

So moving or turning the camera only moves these four corners. Everything else follows from them.

Pixel to point. Let $\mathbf{T}_L$ be the top-left corner, and let $\mathbf{h} = \mathbf{T}_R - \mathbf{T}_L$ and $\mathbf{v} = \mathbf{B}_L - \mathbf{T}_L$ be the vectors along the top and left edges. Pixel (row i, column j) of a $w \times h$ image gets the normalised coordinates $u = j/(w-1)$, $v = i/(h-1)$ and maps to

$$\mathbf{P}_{ij} = \mathbf{T}_L + u\,\mathbf{h} + v\,\mathbf{v}.$$

The primary ray starts at $\mathbf{C}$ and points at $\mathbf{P}_{ij}$.

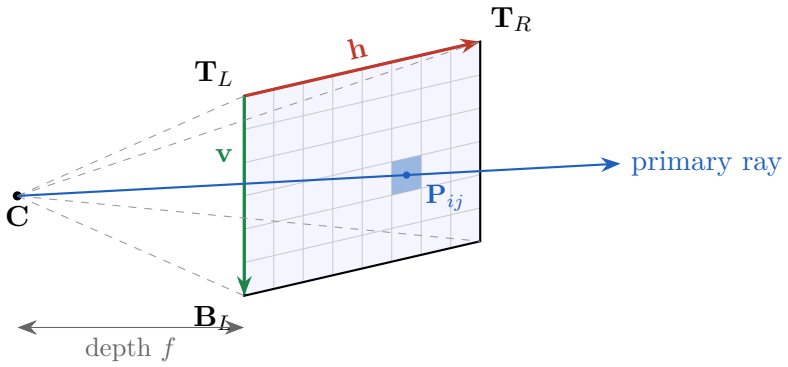


Figure 2: The image plane is a grid of pixels floating at distance f in front of the centre of projection. A pixel's point is found by walking u of the way along the top edge and v of the way down the side.

In short

Compute the four plane corners in world space once. For every pixel, interpolate its point on the plane and shoot a ray from the camera centre through it. Trace that ray (with the full recursion budget) and write the returned colour into the pixel.

Algorithm 1 Generating primary rays

$\mathbf{T}_L, \mathbf{T}_R, \mathbf{B}_L \leftarrow$ plane corners mapped to world space
$\mathbf{h} \leftarrow \mathbf{T}_R - \mathbf{T}_L, \quad \mathbf{v} \leftarrow \mathbf{B}_L - \mathbf{T}_L$
for each row i, each column j **do**
    $u \leftarrow j/(w-1), \quad v \leftarrow i/(h-1)$
    $\mathbf{P} \leftarrow \mathbf{T}_L + u\,\mathbf{h} + v\,\mathbf{v}$
    pixel$[i, j] \leftarrow$ Trace(ray from $\mathbf{C}$ towards $\mathbf{P}$, $D_{\max}$, $n = 1$)
end for

3 Finding the nearest hit

Given a ray, the tracer must find the *closest* object it hits in front of its origin. It does this by brute force: test every object, keep the hit with the smallest positive t, and record the hit point $\mathbf{X} = \mathbf{r}(t)$, the surface normal $\mathbf{N}$ there, and which object was hit. Exact intersection tests can be expensive, so each object first goes through a cheap test.

3.1 Cheap rejection with bounding boxes

Every object stores an *axis-aligned bounding box* (AABB), a box with faces parallel to the coordinate planes that encloses it, enlarged by a tiny margin. A box is the overlap of three *slabs*, one per axis, where a slab is the region between two parallel planes. A ray enters and leaves each slab at two values of t:

$$t_1^{(k)} = \frac{\min_k - o_k}{d_k}, \qquad t_2^{(k)} = \frac{\max_k - o_k}{d_k}, \qquad k \in \{x, y, z\}.$$

The ray is inside the box only while it is inside *all three* slabs at once, that is for $t \in [t_{\text{in}}, t_{\text{out}}]$ with

$$t_{\text{in}} = \max_k \min\left(t_1^{(k)}, t_2^{(k)}\right), \qquad t_{\text{out}} = \min_k \max\left(t_1^{(k)}, t_2^{(k)}\right).$$

If $t_{\text{in}} > t_{\text{out}}$ (the intervals don't overlap) or $t_{\text{out}} \le 0$ (the box is behind the ray), the object cannot be hit and is skipped. If the ray is parallel to a slab, it either lies inside that slab or misses the box outright. The test is *conservative*: it may let through a ray that then misses the object, but it never rejects a real hit.

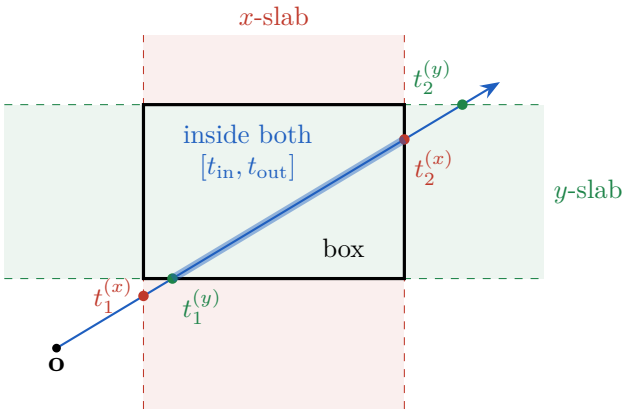


Figure 3: Slab test in 2D. The ray is inside the box exactly while it is inside the x-slab *and* the y-slab. The overlap (shaded on the ray) is non-empty, so this ray is passed on to the exact intersection test.

3.2 Exact intersection per shape

Sphere. A point is on a sphere (centre $\mathbf{c}$, radius r) when $\|\mathbf{x} - \mathbf{c}\|^2 = r^2$. Substituting the ray and writing $\mathbf{L} = \mathbf{o} - \mathbf{c}$ gives a quadratic in t:

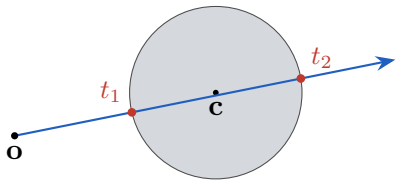
$$\underbrace{\langle \mathbf{d}, \mathbf{d} \rangle}_{a} t^2 + \underbrace{2\langle \mathbf{d}, \mathbf{L} \rangle}_{b} t + \underbrace{\langle \mathbf{L}, \mathbf{L} \rangle - r^2}_{c} = 0, \qquad t_{1,2} = \frac{-b \mp \sqrt{b^2 - 4ac}}{2a}.$$

A negative discriminant means the ray misses. Otherwise the answer is the smaller *positive* root: t_1 if the ray starts outside the sphere, t_2 if it starts inside (which happens for refracted rays travelling through glass). The normal is $(\mathbf{X} - \mathbf{c})/r$, which always points outward.

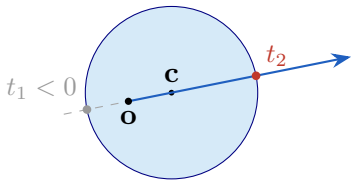
Triangle (Möller–Trumbore). Any point of the plane through $\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2$ can be written as $\mathbf{v}_0 + u\,\mathbf{e}_1 + v\,\mathbf{e}_2$ with edges $\mathbf{e}_1 = \mathbf{v}_1 - \mathbf{v}_0$, $\mathbf{e}_2 = \mathbf{v}_2 - \mathbf{v}_0$. Setting this equal to $\mathbf{o} + t\mathbf{d}$ gives a 3×3 linear system in (u, v, t), which is solved with Cramer's rule using cross and dot products. The point lies inside the triangle exactly when its *barycentric coordinates* satisfy

$$u \ge 0, \qquad v \ge 0, \qquad u + v \le 1,$$

and it counts as a hit only if $t > 0$. If the system's determinant is close to zero, the ray is parallel to the triangle. The normal is the same everywhere on the triangle, $\mathbf{e}_1 \times \mathbf{e}_2$ normalised, so it is computed once.



outside start: take t_1

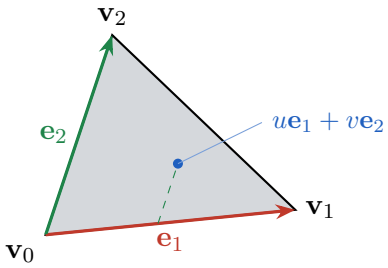


inside start: take t_2

Figure 4: Ray–sphere intersection. The two roots are the entry and exit distances. A root behind the origin ($t \le 0$) is discarded.

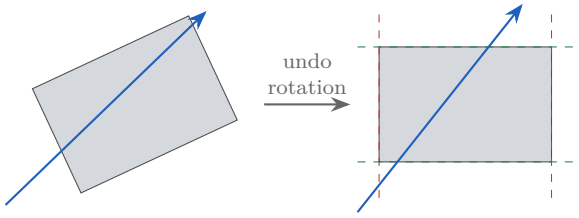
Quad. A quad is split along a diagonal into two triangles. Both are tested and the nearer hit wins.

Block (oriented box). A block can be rotated, so the ray is first moved into the block's own frame (subtract the centre, apply the inverse rotation). There the block is an ordinary axis-aligned box centred at the origin, and the same slab test as above gives the exact hit. This time it uses t_{in} if positive, otherwise t_{out} (ray starting inside). The normal is the face axis along which the local hit point is relatively farthest out, i.e. the largest of $|x|/w,\ |y|/h,\ |z|/d$ with w, h, d the half-sizes. It is given the sign of that coordinate and rotated back to world space.



inside iff $u, v \ge 0,\ u + v \le 1$

(a) Barycentric test.



world block frame

(b) A rotated block becomes an ordinary box.

Figure 5: Flat shapes use barycentric coordinates. Blocks are reduced to the slab test.

In short

Loop over all objects. Skip an object if the ray misses its bounding box. Otherwise run the exact test for its shape and keep the hit if it lies in front of the origin and is nearer than the best hit so far. Afterwards, compute the normal at the winning hit point.

4 When a ray hits nothing: background and glow

4.1 Background

A ray that misses every object takes the background colour. That is either a solid colour or a panoramic (equirectangular) image wrapped around the scene like a sky dome. For the image:

1. Pick a point far along the ray, $\mathbf{S} = \mathbf{C} + 15f\,\hat{\mathbf{d}}$. Using a far point rather than the bare direction gives a little *parallax*: the background shifts slightly when the camera moves, and less so the farther away the point is chosen.

Algorithm 2 Nearest intersection

```
function NearestHit(ray)
    t* ← ∞,  hit ← none
    for each object O do
        if ray misses the bounding box of O then continue
        end if
        t ← exact intersection of ray and O
        if 0 < t < t* then
            t* ← t,  hit ← (O,  X = r(t),  N = normal of O at X)
        end if
    end for
    return hit
end function
```

2. Turn $\mathbf{S}$ into two angles, the *yaw* (around the vertical axis) and the *pitch* (above or below the horizon):

$$\psi = \operatorname{atan2}(S_x, S_z) \in [-\pi, \pi], \qquad \theta = \operatorname{atan2}\left(S_y, \sqrt{S_x^2 + S_z^2}\right) \in [-\tfrac{\pi}{2}, \tfrac{\pi}{2}].$$

3. Rescale the angles to texture coordinates, $u = (\psi + \pi)/2\pi$ (wrapping around horizontally) and $v = (\theta + \pi/2)/\pi$, and read the nearest pixel.

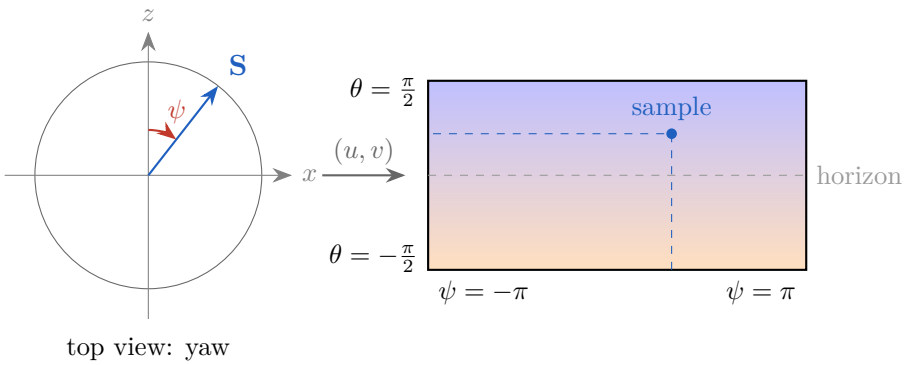


Figure 6: Equirectangular lookup: the yaw angle picks the column and the pitch angle picks the row, with the image wrapping horizontally like a cylinder.

4.2 Glow halos around emissive spheres

Spheres can be *emissive*: they have an emission colour and strength. A surface hit on such a sphere simply adds its emission to its colour. To make lamps look like they glow, the tracer also adds a cheap *halo*, an approximation with no physical basis: every ray picks up some light when it *passes close* to an emitter.

For each emissive sphere (centre $\mathbf{c}$, radius r), find the point on the travelled segment of the ray, $t \in [0, t_{\text{end}}]$, that is closest to the centre. t_{end} is the hit distance, or infinity for a ray that hits nothing:

$$t^* = \operatorname{clamp}\left(\langle \mathbf{c} - \mathbf{o}, \widehat{\mathbf{d}} \rangle,\ 0,\ t_{\text{end}}\right), \qquad \delta = \|\mathbf{r}(t^*) - \mathbf{c}\|.$$

If $\delta < r$, the ray actually hits the sphere and the surface takes care of it. Otherwise the halo contribution decays exponentially with the gap $\delta - r$ between the ray and the silhouette, in the spirit of the Beer–Lambert attenuation law:

$$\text{glow} \mathrel{+}= 0.45 \cdot s \cdot e^{-(\delta - r)/(0.3\,r)} \cdot \text{emission}.$$

Here s is the emission strength. The falloff scales with the radius, so large lamps get wide halos. Glow is added both to rays that miss everything and to rays that hit a surface after passing an emitter.

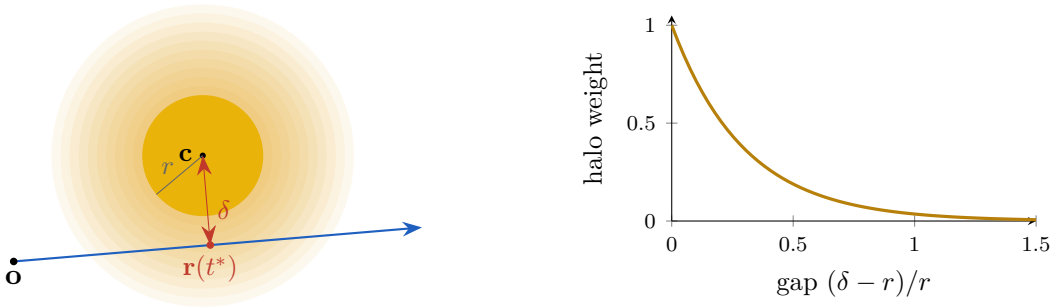


Figure 7: Glow: the closer the ray passes to an emitter's silhouette, the more of its emission it picks up. The weight halves roughly every $0.2r$.

In short

Miss ⇒ background (solid colour or panorama lookup by yaw and pitch) plus the glow of every emitter the ray passed near.

5 Direct lighting: the Phong model

When a ray hits a surface at $\mathbf{X}$ with unit normal $\mathbf{N}$, the first colour computed is the *direct* lighting: light that reaches $\mathbf{X}$ straight from the light sources and bounces once toward the viewer. The tracer uses the classic Phong model, with three kinds of light:

- **ambient**: a constant amount, the same everywhere, that keeps shadows from going completely black;
- **point**: a light at a position $\mathbf{Q}$; the direction to it is $\mathbf{L} = (\mathbf{Q} - \mathbf{X})/\|\mathbf{Q} - \mathbf{X}\|$;
- **directional**: a light infinitely far away (like the sun), given by one fixed direction $\mathbf{L}$.

Before rendering, all intensities k_i are divided by their sum, so together they add up to 1 and a surface can never be over-lit just because the scene has many lights.

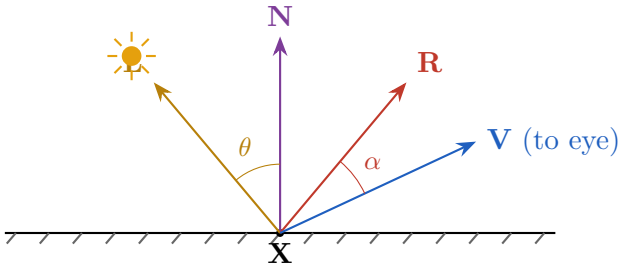


Figure 8: Phong vectors. Diffuse light depends on the angle θ between the normal and the light. The specular highlight depends on the angle α between the mirrored light direction $\mathbf{R}$ and the direction $\mathbf{V}$ to the viewer.

For every non-ambient light i, with shadow factor $b_i \in [0,1]$ (Section 6; 1 means fully lit):

$$I_{\text{diff}} = \sum_{\text{ambient}} k_i \;+\; \sum_{\text{point/dir}} k_i\, b_i\, \max\bigl(0, \langle \mathbf{N}, \mathbf{L}_i \rangle\bigr),$$

$$I_{\text{spec}} = \sum_{\text{point/dir}} k_i\, b_i\, \max\bigl(0, \langle \mathbf{R}_i, \mathbf{V} \rangle\bigr)^s, \qquad \mathbf{R}_i = 2\langle \mathbf{N}, \mathbf{L}_i \rangle \mathbf{N} - \mathbf{L}_i,$$

where $\mathbf{V}$ points from the surface to the camera and the exponent s is the material's shininess (small means a broad, dull highlight; large means a small, sharp one). Both sums are capped at 1. The *albedo* $\mathbf{a}$ (the surface's base colour) comes from the material or, for textured spheres and blocks, from the texture at that point. The direct colour is then

$$\mathbf{c}_{\text{direct}} = \min\bigl(255,\ \mathbf{a}\, I_{\text{diff}} + 255\, I_{\text{spec}}\bigr).$$

The diffuse term *tints* the albedo, while the specular term adds *white*, which is why highlights look like reflections of the light rather than brighter paint.

Where the albedo comes from (texture mapping)

A textured sphere turns its normal into longitude and latitude and reads the texture like a world map. A textured block works out which face was hit, uses the two in-face coordinates as (u, v), and repeats the pattern as many times as requested. Either way, the result simply replaces the flat material colour $\mathbf{a}$ above.

In short

Start with the ambient light. For each other light, get its direction, ask the shadow test how visible it is, then add a diffuse term ($\propto \cos\theta$) and a specular term ($\propto \cos^s \alpha$), both scaled by that visibility. Colour = albedo × diffuse + white × specular.

Algorithm 3 Direct (Phong) shading at a hit point

```
function DIRECT(X, N, object)
    I_d ← 0,  I_s ← 0,  V ← unit vector from X to camera
    for each light i do
        if ambient then I_d ← I_d + k_i; continue
        end if
        b_i ← SHADOW(i, X, N)
        if b_i ≈ 0 then continue                          ▷ fully blocked
        end if
        L ← direction to light i
        I_s ← I_s + k_i b_i max(0, ⟨R, V⟩)^s               ▷ only for shiny materials
        I_d ← I_d + k_i b_i max(0, ⟨N, L⟩)
    end for
    return min(255, albedo(X) · min(I_d, 1) + 255 min(I_s, 1))
end function
```

6 Shadow rays

6.1 The idea

A point is in shadow with respect to a light if something sits between it and that light. To find out, the tracer shoots a **shadow ray** from the hit point $\mathbf{X}$ toward the light and checks whether any *other* object is hit before the ray gets there. An object never shadows itself. Since all the shapes are convex, that is safe.

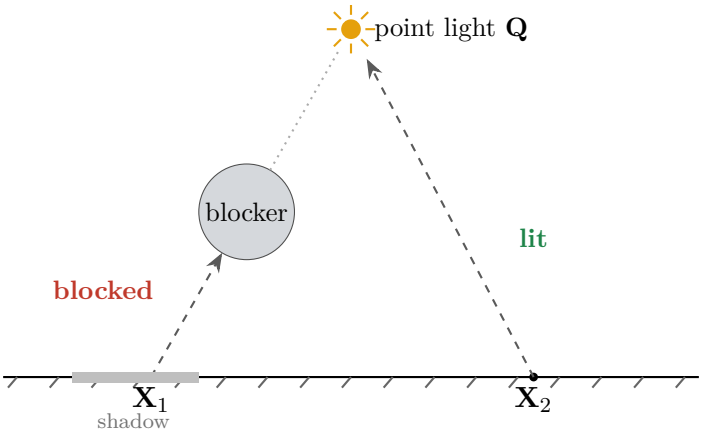


Figure 9: Shadow rays. From $\mathbf{X}_1$ the ray to the light hits the blocker at a distance shorter than the light's, so $\mathbf{X}_1$ is in shadow. From $\mathbf{X}_2$ the path is clear.

6.2 Avoiding shadow acne

The hit point $\mathbf{X}$ is computed in floating point, so it can land a hair *below* the surface. A shadow ray started there would immediately hit the very surface it leaves and shadow itself, which shows up as dark speckles ("shadow acne"). The fix is to push the shadow ray's origin a tiny distance ε off the surface, along the normal *on the light's side*:

$$\mathbf{o}_{\text{shadow}} = \mathbf{X} + \varepsilon \, \operatorname{sign}\big(\langle \mathbf{N}, \mathbf{L} \rangle\big) \, \mathbf{N}.$$

Choosing the side by the sign of $\langle \mathbf{N}, \mathbf{L} \rangle$ makes this work whether the light is in front of the surface or behind it (for example a point inside a glass object). The same trick is used for reflected and refracted rays, with a larger offset.

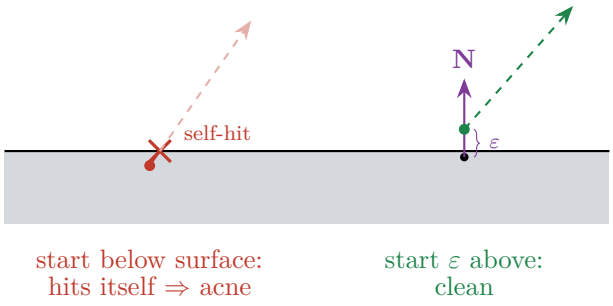


Figure 10: Offsetting the origin along the normal prevents self-shadowing.

6.3 How dark is the shadow? (soft-shadow heuristic)

Real lights have a size, so shadows have soft edges. This tracer imitates that softness with no extra rays at all. Instead of a hard "lit / not lit", the shadow test returns a *brightness* $b \in [0, 1]$ (1: no shadow, 0: black).

Point lights. Let d be the distance from the (offset) origin to the light and t_{occ} the distance to the *nearest* blocker on the shadow ray. Only blockers with $0 < t_{\text{occ}} < d$ count: something behind the light cannot cast a shadow. If there is a blocker,

$$b = \langle \mathbf{N}, \mathbf{L} \rangle \cdot \frac{t_{\text{occ}}}{d} \quad \text{(clamped to } [0, 1]\text{)}.$$

- The ratio t_{occ}/d makes a blocker *close to the surface* cast a dark, solid shadow, while a blocker *close to the light* casts a faint one. This matches what happens with a real light of some size: the further the blocker is from the surface, the more of the light can reach around it.
- The factor $\langle \mathbf{N}, \mathbf{L} \rangle$ darkens shadows further on surfaces that face away from the light.

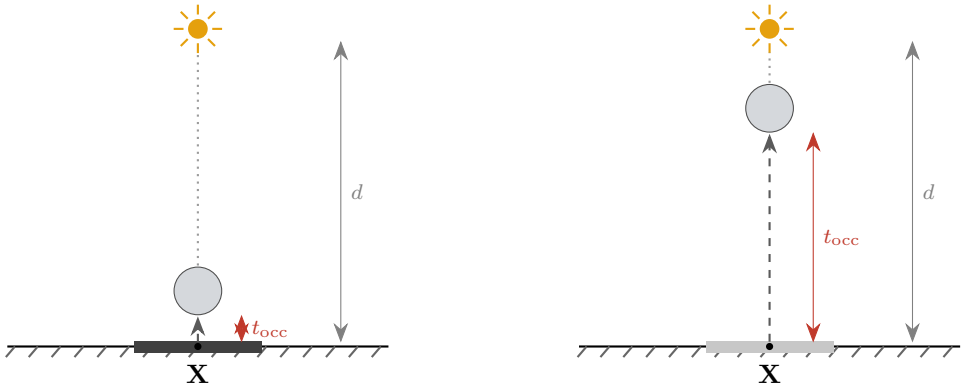


Figure 11: Same light, same blocker size, different positions. Scaling the brightness by t_{occ}/d gives shadows that fade as the blocker moves toward the light.

Directional lights. A directional light is infinitely far away, so there is no distance to compare against. Any blocker hit at $t > 0$ counts, and the brightness keeps only the orientation part, $b = \max(0, \langle \mathbf{N}, \mathbf{L} \rangle)$. One exception: if the surface faces away from the shadow ray *and* so does the blocker's surface where the shadow ray hits it, the point lies inside or against that other object, and the blocker is ignored.

Ambient light is never shadowed.

In short

For each point or directional light: offset the origin off the surface, toward the light. Shoot a ray to the light and look for the nearest other object between the surface and the light. No blocker means $b = 1$. A blocker means a partial brightness: *orientation* × *how far along the blocker sits* (point lights), or just orientation (directional lights).

7 Reflected rays

A mirror sends light back out at the same angle it came in, on the other side of the normal. With a unit incoming direction $\mathbf{I}$ (pointing *toward* the surface) and a unit normal $\mathbf{N}$ on the incoming side, the reflected direction is

$$\boxed{\mathbf{R} = \mathbf{I} - 2\langle \mathbf{I}, \mathbf{N} \rangle \, \mathbf{N}}$$

The reason: $\langle \mathbf{I}, \mathbf{N} \rangle \mathbf{N}$ is the part of $\mathbf{I}$ that goes into the surface. Subtracting it once leaves only the part along the surface. Subtracting it a second time flips the into-surface part, so the ray now leaves the surface (Figure 12).

The reflected ray starts at $\mathbf{X}$ pushed out by a small ε along the normal on the incoming side, which stops it from hitting the surface it is leaving. It is traced recursively with one less unit of depth, and it travels in the same medium as the incoming ray.

Algorithm 4 Shadow factor for one light

function SHADOW(light, $\mathbf{X}, \mathbf{N}$)
  $\mathbf{L} \leftarrow$ unit direction from $\mathbf{X}$ toward the light
  $\mathbf{o} \leftarrow \mathbf{X} + \varepsilon\,\mathrm{sign}(\langle \mathbf{N}, \mathbf{L} \rangle)\,\mathbf{N}$ ▷ acne guard
  if point light at $\mathbf{Q}$ **then**
    $d \leftarrow \|\mathbf{Q} - \mathbf{o}\|$; $t_{\mathrm{occ}} \leftarrow$ nearest hit in $(0, d)$ on ray $(\mathbf{o}, \mathbf{L})$, ignoring self
    if no hit **then return** 1
    end if
    return clamp $\left(\langle \mathbf{N}, \mathbf{L} \rangle \cdot t_{\mathrm{occ}}/d,\ 0, 1\right)$
  else ▷ directional
    if no other object hit along $(\mathbf{o}, \mathbf{L})$ (excluding "inside" cases) **then return** 1
    end if
    return clamp$(\langle \mathbf{N}, \mathbf{L} \rangle, 0, 1)$
  end if
end function

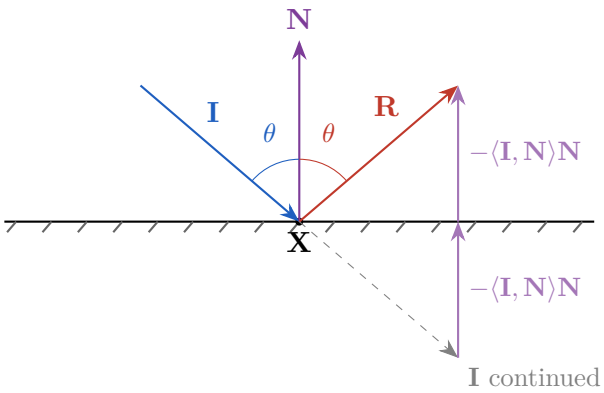


Figure 12: Reflection as "take away the into-surface part twice". The angle to the normal is the same on both sides.

When is a reflected ray traced? Whenever the surface is mirror-like (reflectivity $\rho > 0$) *or* transparent ($\tau > 0$). Transparent surfaces reflect too, more so at grazing angles (Fresnel effect, next section), so they need the reflected colour even if $\rho = 0$.

In short

Mirror the incoming direction about the normal, start just above the surface, and recursively trace. The returned colour is later blended in with weight $\rho + \tau R_F$, where R_F is the Fresnel reflectance of Section 8.

Algorithm 5 Reflected ray

$\mathbf{N} \leftarrow$ normal flipped to face the incoming ray
$\mathbf{R} \leftarrow \mathbf{I} - 2\langle \mathbf{I}, \mathbf{N} \rangle \mathbf{N}$
$\mathbf{c}_{\mathrm{refl}} \leftarrow \mathrm{TRACE}\left((\mathbf{X} + \varepsilon\mathbf{N},\ \mathbf{R}),\ \mathrm{depth} - 1,\ n_1\right)$

8 Refracted rays

8.1 Snell's law

Light slows down in glass or water. The *index of refraction* n says by how much ($n = 1$ for air, ≈ 1.5 for glass). When a ray crosses from a medium with index n_1 into one with n_2, it bends according to Snell's law:

$$n_1 \sin\theta_1 = n_2 \sin\theta_2.$$

Going into a denser medium ($n_2 > n_1$) bends the ray *toward* the normal. Going into a thinner one bends it *away*.

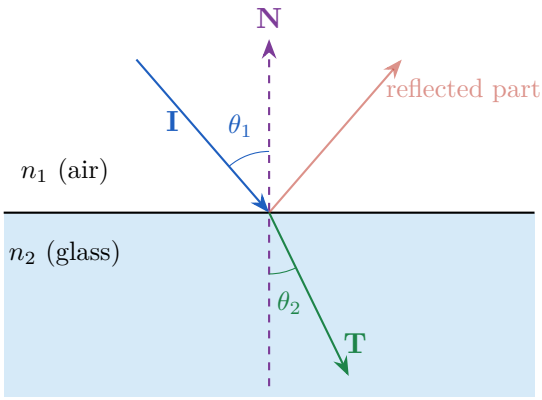


Figure 13: Entering a denser medium: the transmitted ray **T** bends toward the normal, so $\theta_2 < \theta_1$. Some light is always reflected as well.

In vector form, with $\eta = n_1/n_2$, **N** facing the incoming side and $\cos\theta_1 = -\langle \mathbf{N}, \mathbf{I} \rangle$:

$$k = 1 - \eta^2(1 - \cos^2\theta_1), \qquad \boxed{\mathbf{T} = \eta\,\mathbf{I} + (\eta\cos\theta_1 - \sqrt{k})\,\mathbf{N}}$$

Here $k = \cos^2\theta_2$ follows from Snell's law. The first term keeps the along-surface part of **I**, scaled by η. The second term sets the into-surface part so that the result has unit length.

8.2 Total internal reflection

If $k < 0$, there is no real θ_2: the ray is trying to leave a dense medium at too shallow an angle (past the *critical angle* $\theta_c = \arcsin(n_2/n_1)$). Then no light gets through and the surface acts as a perfect mirror. This is **total internal reflection** (TIR), the reason diamonds sparkle and the edges of a glass ball look silvery. In this case the tracer traces no refracted ray and gives all of the transmitted share to the reflected ray.

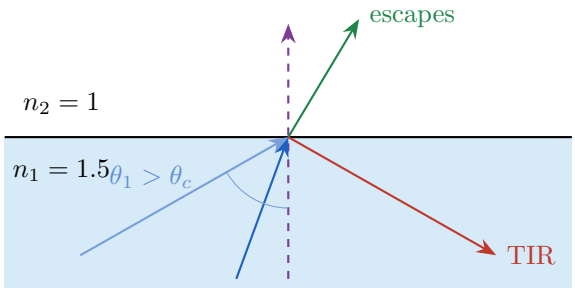


Figure 14: Leaving glass: steep rays escape (bent away from the normal), but past the critical angle ($\approx 41.8°$ for glass) all light reflects back inside.

8.3 Entering or leaving? Orienting the normal and finding n_1, n_2

Object normals always point *outward*. The sign of $\langle \mathbf{N}, \mathbf{I} \rangle$ tells the tracer which side the ray comes from:

- $\langle \mathbf{N}, \mathbf{I} \rangle < 0$: the ray is **entering** the object. Use $\mathbf{N}$ as it is. n_1 is the medium the ray is travelling in and n_2 the object's index.
- $\langle \mathbf{N}, \mathbf{I} \rangle > 0$: the ray is **leaving** the object. Flip the normal ($-\mathbf{N}$) so it faces the incoming side. n_1 is the object's index and n_2 that of whatever surrounds the object.

Every ray carries the index of the medium it is in, which starts at 1 (air) for primary rays. A refracted child ray carries n_2; a reflected child ray carries n_1. This is how the tracer knows it is inside glass.

What surrounds the object? Objects can be nested, for example an air bubble inside a glass block. To find the medium on the outside of a surface, the tracer places a *probe* point a little outside the surface, along the outward normal, and asks each other closed solid (spheres and blocks) whether the probe is inside it. If one contains it, its index is the surrounding index; otherwise it is air ($n = 1$). A cheap bounding-box check comes first here too.

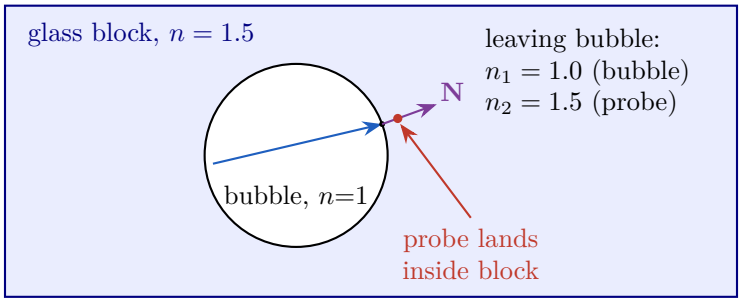


Figure 15: Nested media. A ray travelling inside an air bubble reaches the bubble's surface. The probe just outside that surface lands inside the glass block, so the ray continues into $n_2 = 1.5$, not into air.

8.4 How much is reflected? Fresnel and Schlick

A transparent surface does not let all its light through. The fraction it reflects, R_F, rises sharply toward grazing angles. That is why a lake mirrors the far shore but is see-through at your feet. The exact Fresnel equations are expensive, so the tracer uses **Schlick's approximation**:

$$R_0 = \left(\frac{n_1 - n_2}{n_1 + n_2} \right)^2, \qquad R_F(\theta_1) = R_0 + (1 - R_0)\,(1 - \cos\theta_1)^5.$$

R_0 is the reflectance when looking straight on (about 4% for air/glass).

8.5 Tint

Coloured glass shades the light that passes through it. A tint strength $\kappa \in [0, 1]$, scaled by the transparency τ, blends each colour channel of the refracted colour toward the object's own colour $\mathbf{a}$:

$$c_{\text{refr}} \leftarrow c_{\text{refr}} \cdot \left[(1 - \tau\kappa) + \tau\kappa \, \frac{a}{255} \right] \quad \text{(per channel)}.$$

With $\kappa = 0$ the glass is perfectly clear. With $\kappa = 1$ and full transparency, each channel of what is seen through it is multiplied by the glass's colour.

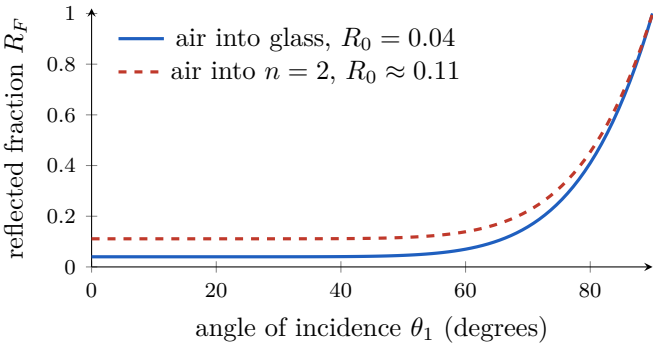


Figure 16: Schlick's curve: almost flat until about 60°, then climbing fast to full reflection at grazing incidence.

In short

Find out whether the ray is entering or leaving. Orient the normal and pick (n_1, n_2), probing for the surrounding medium if needed. Compute Schlick's R_F. Bend the ray with vector Snell. If $k < 0$ it is TIR, so no refracted ray and the reflected one takes over. Otherwise start just past the surface along $\mathbf{T}$, trace recursively in medium n_2 and tint the result.

Algorithm 6 Refracted ray

if $\langle \mathbf{N}, \mathbf{I} \rangle < 0$ **then** ▷ entering
  $\mathbf{N}' \leftarrow \mathbf{N}$, $n_1 \leftarrow n_{\text{ray}}$, $n_2 \leftarrow n_{\text{object}}$
else ▷ leaving
  $\mathbf{N}' \leftarrow -\mathbf{N}$, $n_1 \leftarrow n_{\text{object}}$, $n_2 \leftarrow \textsc{SurroundingIndex}(\mathbf{X})$
end if
$\eta \leftarrow n_1 / n_2$, $\cos\theta_1 \leftarrow -\langle \mathbf{N}', \mathbf{I} \rangle$, $R_F \leftarrow \text{Schlick}(n_1, n_2, \cos\theta_1)$
$k \leftarrow 1 - \eta^2 (1 - \cos^2\theta_1)$
if $k < 0$ **then**
  TIR: no refracted ray, move its weight to reflection
else
  $\mathbf{T} \leftarrow \eta \mathbf{I} + (\eta \cos\theta_1 - \sqrt{k})\mathbf{N}'$
  $\mathbf{c}_{\text{refr}} \leftarrow \textsc{Trace}\big((\mathbf{X} + \varepsilon \mathbf{T},\ \mathbf{T}),\ \text{depth} - 1,\ n_2\big)$
  apply tint toward the object colour
end if

9 Blending everything at a hit point

A surface can be part matte, part mirror and part glass at once. Its material gives a reflectivity $\rho \in [0, 1]$ and a transparency $\tau \in [0, 1]$. Together with the Fresnel reflectance R_F, these decide how much each colour contributes:

$w_t = \tau\,(1 - R_F)$	transmitted (refracted) share
$w_r = \rho + \tau\, R_F$	reflected share: mirror part + glass reflection
$w_d = 1 - \min(w_r + w_t,\ 1)$	whatever remains goes to direct lighting

Under total internal reflection, $w_t = 0$ and $w_r \leftarrow \min(1, w_r + \tau)$. The direct colour of a transparent surface is also dimmed by $(1 - \tau)$ before blending, because a clear object has little "paint" of its own. The final colour is

$$\boxed{\mathbf{c} = w_d\,(1 - \tau)\,\mathbf{c}_{\text{direct}} + w_r\,\mathbf{c}_{\text{refl}} + w_t\,\mathbf{c}_{\text{refr}} + \text{glow}}$$

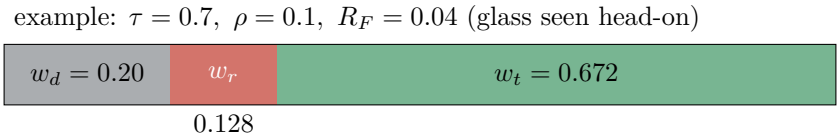


Figure 17: How one pixel's colour is split between a mostly transparent, slightly mirrored surface's three sources.

When does the recursion stop? At a hit, no child rays are spawned (the colour is just the direct light plus any emission) if

- the depth budget is used up, or
- the surface is neither reflective nor transparent ($\rho \approx 0$ and $\tau \approx 0$).

Otherwise, every hit spawns a reflected ray and, if transparent, a refracted ray as well. The rays therefore form a binary *ray tree*, with shadow rays as leaves hanging off every node (Figure 18). The tree has at most 2^D leaves, which is why the depth budget D is the main control over speed.

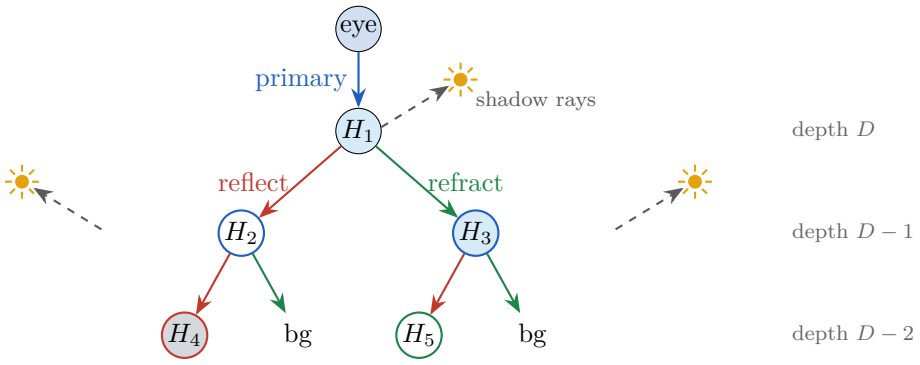


Figure 18: The ray tree for one pixel. Each reflective/transparent hit branches into a reflected (red) and a refracted (green) child. Every hit also sends shadow rays to the lights for its direct term. "bg" marks rays that escape to the background. Colours flow *up* the tree and are blended at each node.

10 Post-processing: gamma correction

Once every pixel has a colour, the image is gamma-corrected. Each 8-bit value is remapped through

$$c' = 255 \left(\frac{c}{255} \right)^{\gamma}.$$

The map is precomputed into a 256-entry lookup table, so applying it is just one array read per channel. $\gamma < 1$ lifts the dark tones (useful because the weights above tend to darken the image), while $\gamma > 1$ deepens them.

11 Putting it all together

The pieces now fit into one recursive procedure. RENDER runs once. TRACE is called for every primary ray and then calls itself for reflected and refracted rays. DIRECT and SHADOW handle the lighting at each hit.

Algorithm 7 The complete freestyle ray tracer: rendering and tracing

function RENDER(scene, camera, $D_{\max}$, γ)
  divide every light intensity by the sum of all intensities
  collect the emissive spheres (for glow)
  $\mathbf{T}_L, \mathbf{T}_R, \mathbf{B}_L \leftarrow$ image-plane corners in world space; $\mathbf{h} \leftarrow \mathbf{T}_R - \mathbf{T}_L$, $\mathbf{v} \leftarrow \mathbf{B}_L - \mathbf{T}_L$
  for each pixel (i, j) **do**
    $\mathbf{P} \leftarrow \mathbf{T}_L + \frac{j}{w-1}\mathbf{h} + \frac{i}{h-1}\mathbf{v}$
    image$[i, j] \leftarrow$ TRACE(ray($\mathbf{C} \to \mathbf{P}$), $D_{\max}$, 1)
  end for
  apply gamma $c \mapsto 255(c/255)^{\gamma}$ to every channel
end function

function TRACE(ray $(\mathbf{o}, \mathbf{I})$, depth, n_{ray})
  $(O,\ \mathbf{X},\ \mathbf{N},\ t_{\text{hit}}) \leftarrow$ NEARESTHIT(ray) ▷ AABB cull + exact tests
  if no hit **then**
    return BACKGROUND($\mathbf{I}$) + GLOW(ray, ∞)
  end if
  $\rho, \tau \leftarrow$ reflectivity and transparency of O
  $\mathbf{c}_d \leftarrow (1 - \tau) \cdot$ DIRECT($\mathbf{X}, \mathbf{N}, O$) ▷ Phong + shadow rays
  if depth $= 0$ **or** ($\rho \approx 0$ **and** $\tau \approx 0$) **then**
    return $\mathbf{c}_d +$ emission(O) ▷ leaf of the ray tree
  end if
  — orientation —
  if $\langle \mathbf{N}, \mathbf{I} \rangle < 0$ **then** $\mathbf{N}' \leftarrow \mathbf{N}$; $n_1 \leftarrow n_{\text{ray}}$; $n_2 \leftarrow n_O$ ▷ entering
  else $\mathbf{N}' \leftarrow -\mathbf{N}$; $n_1 \leftarrow n_O$; $n_2 \leftarrow$ SURROUNDINGINDEX($\mathbf{X}$) ▷ leaving
  end if
  $\eta \leftarrow n_1/n_2$, $\cos\theta_1 \leftarrow -\langle \mathbf{N}', \mathbf{I} \rangle$
  — reflection —
  $\mathbf{R} \leftarrow \mathbf{I} - 2\langle \mathbf{I}, \mathbf{N}' \rangle \mathbf{N}'$
  $\mathbf{c}_r \leftarrow$ TRACE($(\mathbf{X} + \varepsilon\mathbf{N}',\ \mathbf{R})$, depth $- 1$, n_1)
  — refraction —
  $R_F \leftarrow R_0 + (1 - R_0)(1 - \cos\theta_1)^5, \quad R_0 = \left(\frac{n_1 - n_2}{n_1 + n_2}\right)^2$
  $w_t \leftarrow \tau(1 - R_F)$, $w_r \leftarrow \rho + \tau R_F$, $\mathbf{c}_t \leftarrow \mathbf{0}$
  if $\tau > 0$ **then**
    $k \leftarrow 1 - \eta^2(1 - \cos^2\theta_1)$
    if $k < 0$ **then** ▷ total internal reflection
      $w_t \leftarrow 0$, $w_r \leftarrow \min(1,\ w_r + \tau)$
    else
      $\mathbf{T} \leftarrow \eta\mathbf{I} + (\eta\cos\theta_1 - \sqrt{k})\mathbf{N}'$
      $\mathbf{c}_t \leftarrow$ TRACE($(\mathbf{X} + \varepsilon\mathbf{T},\ \mathbf{T})$, depth $- 1$, n_2)
      $\mathbf{c}_t \leftarrow \mathbf{c}_t \odot \left[(1 - \tau\kappa) + \tau\kappa\, \mathbf{a}_O/255\right]$ ▷ tint
    end if
  end if
  — blend —
  $w_d \leftarrow 1 - \min(w_r + w_t, 1)$
  return $w_d\, \mathbf{c}_d + w_r\, \mathbf{c}_r + w_t\, \mathbf{c}_t +$ GLOW(ray, t_{hit})
end function

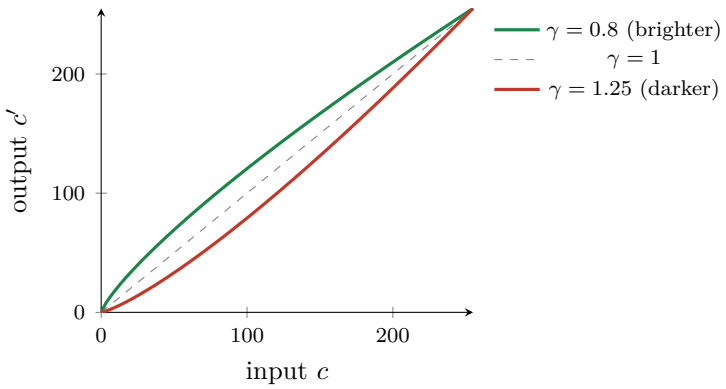


Figure 19: Gamma curves.

Algorithm 8 The complete ray tracer, continued: lighting at a hit

function DIRECT($\mathbf{X}, \mathbf{N}, O$)
  $I_d \leftarrow \sum_{\text{ambient}} k_i$, $I_s \leftarrow 0$, $\mathbf{V} \leftarrow$ unit vector from $\mathbf{X}$ to camera
  for each point or directional light i **do**
    $b \leftarrow$ SHADOW($i, \mathbf{X}, \mathbf{N}$); **if** $b \approx 0$ **continue**
    $I_d \leftarrow I_d + k_i\, b \max(0, \langle \mathbf{N}, \mathbf{L}_i \rangle)$; $I_s \leftarrow I_s + k_i\, b \max(0, \langle \mathbf{R}_i, \mathbf{V} \rangle)^s$
  end for
  return $\min\left(255,\ \text{albedo}_O(\mathbf{X}) \min(I_d, 1) + 255 \min(I_s, 1)\right)$
end function

function SHADOW(light i, $\mathbf{X}, \mathbf{N}$)
  $\mathbf{o} \leftarrow \mathbf{X} + \varepsilon\, \text{sign}(\langle \mathbf{N}, \mathbf{L}_i \rangle)\mathbf{N}$
  find nearest other object on ray $(\mathbf{o}, \mathbf{L}_i)$, before the light if it is a point light
  if none **then return** 1
  end if
  if point light **then return** $\text{clamp}\left(\langle \mathbf{N}, \mathbf{L}_i \rangle \cdot t_{\text{occ}}/d,\ 0, 1\right)$
  else return $\text{clamp}(\langle \mathbf{N}, \mathbf{L}_i \rangle, 0, 1)$
  end if
end function

Reading the algorithm as a story. For each pixel, a ray leaves the eye. It either escapes to the sky (background plus any glow it passed through) or lands on a surface. There it gathers light from the lamps, with each lamp dimmed by its shadow ray. If the surface is plain, that is the answer. If it is shiny or glassy, the ray splits: one copy bounces like a mirror, the other bends into or out of the material, and each copy repeats the whole story with one less unit of depth. On the way back up, the three colours (direct, reflected, refracted) are mixed with weights set by the material and by Fresnel's law, so glass mirrors more at grazing angles and passes more light head-on.

A Notes from reading the code

The sections above describe what each stage is *meant* to do. While reading the implementation, a few places stood out where the code seems to behave differently. They are worth double-checking:

1. **Refraction indices on entry/exit were swapped (fixed).** Previously, on *entering* both n_1 and n_2 were set to the ray's current index, so $\eta = 1$ and the ray passed into the object without bending. On *leaving*, the pair was reversed, so the ray bent as if it were entering, and total internal reflection could never happen. The code now uses the pairing from Section 8 (entering: $n_{\text{ray}} \to n_O$; leaving: $n_O \to n_{\text{surround}}$). A glass ball now inverts the scene behind it like a real

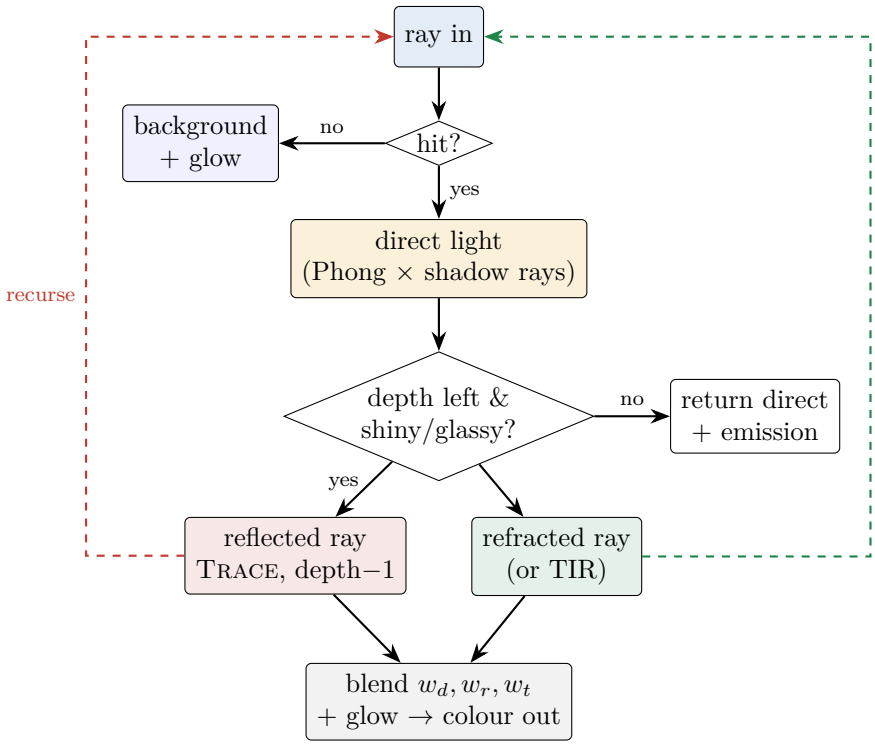


Figure 20: Control flow of TRACE. The dashed arrows are the recursive calls.

lens, and TIR shows up inside glass (see `docs/refraction_fix/`).

2. **Directional-light shadow rays point away from the light.** The diffuse term treats the stored direction as pointing *toward* the light ($\langle \mathbf{N}, \mathbf{L} \rangle$), and the origin offset picks the hemisphere with the same sign. The shadow ray, however, is shot along the *negated* direction, and the brightness also uses the negated direction, so it is ≤ 0 (fully dark) on faces turned toward the light. Shooting the shadow ray along the stored direction and using $\langle \mathbf{N}, \mathbf{L} \rangle$ would match the point-light logic.

3. **Schlick when leaving a dense medium.** For $n_1 > n_2$, the approximation is more accurate with $\cos\theta_2$ (the transmitted angle) in place of $\cos\theta_1$. Otherwise R_F stays low right up to the critical angle. Since TIR is handled separately, this is a quality issue rather than a bug.

4. **Direct light on transparent surfaces is attenuated twice**: once by $(1 - \tau)$ and once more by $w_d \approx 1 - \rho - \tau$. For $\tau = 0.7$ the direct term ends up weighted by about 0.09. That may be intended (to keep glass clear), but one of the two factors is probably enough.

5. **Emission is only added at leaves.** An emissive sphere that is also reflective or transparent goes down the recursive branch and loses its own emission (glow along the ray is still added).